

# Procedimientos almacenados

## *5. Procedimientos almacenados*

# Qué es un procedimiento almacenado

## Hasta ahora: la lógica en Java

pide las filas → las recorre → decide qué hacer

## Ahora: la lógica en el motor

tu app lo invoca → el motor lo ejecuta → sin traer nada

## PL/pgSQL: el lenguaje procedural de PostgreSQL

```
CREATE OR REPLACE PROCEDURE
    nombre_procedimiento(parametro TIPO)
LANGUAGE plpgsql
AS $$
BEGIN
    -- lógica con SQL normal aquí dentro
END;
$$;
```

## Definición

Código SQL con lógica (variables, condiciones, bucles) que se guarda y se ejecuta dentro del gestor. Lo defines una vez y se invoca por su nombre.

# Procedimiento vs. función, y sus parámetros

## Función

Siempre devuelve un valor y se puede usar dentro de una consulta: `SELECT mi_funcion(x)`.

## Procedimiento

No tiene por qué devolver nada y se invoca aparte con `CALL`. Encaja mejor para «ejecutar una acción» que para «calcular un valor».

## Parámetros por posición y de entrada (IN)

El primer valor que envías desde Java rellena el primer parámetro, el segundo el segundo... El procedimiento puede leerlos, pero no modificarlos y devolvértelos.

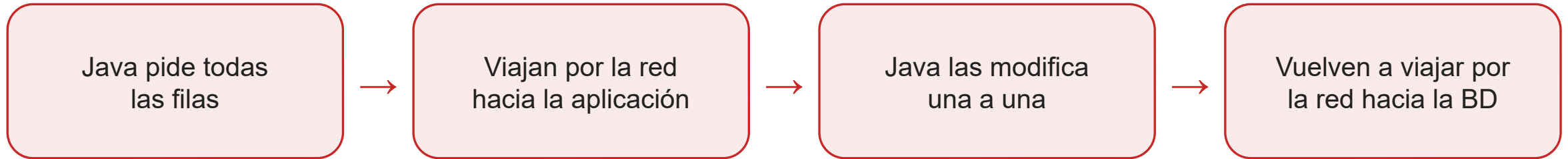
## Si inviertes el orden al invocarlo, el motor no avisa

Asigna cada valor al parámetro equivocado y el error solo aparece cuando compruebas el resultado.

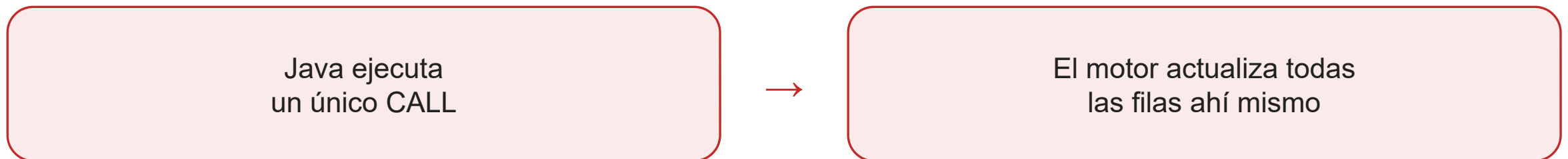
# Cuándo tiene sentido usarlos

Cuando una operación conviene que ocurra de forma atómica y cerca de los datos, como un ajuste masivo de precios que afecta a muchas filas a la vez.

## Sin procedimiento almacenado



## Con procedimiento almacenado



Hacerlo dentro del motor evita traer todas las filas a Java y devolverlas, con todo el tráfico de red que eso implica.

# El inconveniente: la lógica en dos sitios

**La lógica queda repartida entre tu código Java y la base de datos**

Más difícil de versionar junto al resto del proyecto y de probar con las mismas herramientas.

|                          | Ventaja                                               | Coste                                                                |
|--------------------------|-------------------------------------------------------|----------------------------------------------------------------------|
| Procedimiento almacenado | Una sola operación atómica, sin ida y vuelta por fila | Lógica repartida en dos sitios; más difícil de versionar y de probar |
| Bucle en Java            | Todo el código en un solo sitio, fácil de testear     | Un viaje de red por cada fila leída y modificada                     |

# Cómo se invocan desde Java

## CallableStatement

La vía estándar de JDBC: el primo de PreparedStatement pensado para procedimientos. Solo para que sepas que existe.

## JdbcTemplate

El ayudante de Spring sobre JDBC puro: sin abrir conexión, cerrar recursos ni gestionar excepciones a mano. No llega a ser un ORM.

### Una sola línea:

```
jdbcTemplate.update("CALL ajustar_precio_editorial(?, ?)", editorialId, porcentaje);
```

JdbcTemplate abre la conexión, prepara la sentencia con parámetros (igual de seguros frente a inyección SQL que el PreparedStatement que ya conoces), la ejecuta y cierra todo por ti.

# Ejemplo: ajustar\_precio\_editorial

Una editorial sube (o baja) el precio de todo su catálogo en un porcentaje: una operación masiva, atómica y cerca de los datos.

## 1. El procedimiento en PostgreSQL

```
CREATE OR REPLACE PROCEDURE
    ajustar_precio_editorial(
        p_editorial_id BIGINT,
        p_porcentaje NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE libro
    SET precio = ROUND(
        precio * (1 + p_porcentaje / 100.0), 2)
    WHERE editorial_id = p_editorial_id;
END;
$$;
```

### De una sola vez

Sube o baja el precio de todos los libros de una editorial en el porcentaje indicado.

### Por qué ROUND(..., 2)

precio tiene precisión 10,2. Sin redondear, el cálculo podría dejar más decimales de los que la columna admite.

# La invocación desde el service

## 2. El método del service

```
@Service
@RequiredArgsConstructor
public class EditorialService {
    private final EditorialRepository editorialRepository;
    private final JdbcTemplate jdbcTemplate;

    @Transactional
    public void ajustarPrecio(Long editorialId,
                              BigDecimal porcentaje) {
        if (!editorialRepository.existsById(editorialId)) {
            throw new ResponseStatusException(
                HttpStatus.NOT_FOUND,
                "Editorial no encontrada");
        }
        jdbcTemplate.update(
            "CALL ajustar_precio_editorial(?, ?)",
            editorialId, porcentaje);
    }
}
```

### Un bean más

JdbcTemplate se inyecta como cualquier otro, sin configuración: Spring Boot lo crea al detectar un DataSource.

### JPA y JDBC conviven

existsById sigue siendo Spring Data JPA: solo la parte que conviene ejecutar cerca de los datos va por JdbcTemplate.

### @Transactional

Una lectura y una escritura: el método entero es una sola unidad todo-o-nada.

# ¿Por qué un procedimiento y no un bucle en Java?

## Desde Java

- 1 Recuperar todos los libros de la editorial.
- 2 Recorrerlos con un bucle.
- 3 Actualizar su precio uno por uno.

## Con el procedimiento

La lógica ya está guardada dentro de PostgreSQL y la aplicación solo la invoca con CALL. El gestor actualiza todos los libros sin enviarlos antes a Java ni recorrerlos uno a uno.

La aplicación solicita una operación, pero la lógica que la ejecuta está almacenada dentro de la propia base de datos.

## Todavía falta un paso

Como cualquier otra operación de tu API, aún no es alcanzable desde fuera: falta un endpoint en el controller que llame a `ajustarPrecio(...)` y lo exponga por HTTP.

# Antes de cerrar el tema: consultas derivadas

En «Operaciones CRUD y transacciones» viste findByEditorialId como idea suelta. Aquí, la convención completa: Spring Data JPA lee el nombre del método pieza a pieza y lo traduce a una consulta.

| Método (sobre LibroRepository)                       | Qué genera                                    |
|------------------------------------------------------|-----------------------------------------------|
| findByTitulo(String titulo)                          | WHERE titulo = ?                              |
| findByPrecioLessThan(BigDecimal precio)              | WHERE precio < ?                              |
| findByTituloContainingIgnoreCase(String fragmento)   | WHERE LOWER(titulo) LIKE LOWER('%fragmento%') |
| findByEditorialIdOrderByPrecioDesc(Long editorialId) | WHERE editorial_id = ? ORDER BY precio DESC   |
| existsByTitulo(String titulo)                        | como findByTitulo, pero devuelve boolean      |
| countByEditorialId(Long editorialId)                 | devuelve long: cuántas filas, no cuáles       |
| deleteByEditorialId(Long editorialId)                | DELETE FROM libro WHERE editorial_id = ?      |

# Las piezas se combinan entre sí

```
List<Libro> findByTituloContainingIgnoreCaseOrderByPrecioAsc(String fragmento);
```

**findBy**

Busca

**TituloContainingIgnoreCase**

El campo titulo contiene el  
fragmento, sin distinguir  
mayúsculas

**OrderByPrecioAsc**

Resultado ordenado por precio  
ascendente

## Un nombre mal escrito falla al arrancar, no al llamarlo

Spring Data JPA valida cada método derivado contra los campos de la entidad al arrancar. Con `findByTitle` en vez de `findByTitulo`, la aplicación ni arranca: `PropertyReferenceException` señala qué parte del nombre no ha sabido interpretar.

# Ideas clave

1

Un procedimiento almacenado es código con lógica que vive y se ejecuta dentro del gestor, no en la aplicación.

2

PostgreSQL usa PL/pgSQL. Un procedimiento se invoca con `CALL`; una función se puede usar dentro de un `SELECT`.

3

Los parámetros van por posición y son `IN` por defecto: invertir el orden no da error, asigna mal los valores.

4

Permite operar dentro de la base de datos sin cargar y recorrer filas una a una desde Java.

5

`JdbcTemplate` es el ayudante de Spring sobre JDBC: parámetros seguros y convive con los repositorios JPA.

6

Las consultas derivadas por nombre generan SQL sin escribirlo, pero un nombre mal escrito falla al arrancar, no al llamarlo.